Memory Efficient Doubly Linked List – Insert At Beginning And Time Complexity.

Program :

```
typedef struct XNode
{
    int data;
    struct XNode *npx; // XOR of next and prev pointer
} Node;
Node *head;
Node *XOR(Node *a, Node *b)
{
    return (Node *)((uintptr_t)(a) ^ (uintptr_t)(b));
}
void insertAtBeginning(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }

    newNode->data = data;

    // Logic:
    // New Node's npx = NULL ^ current_head
    newNode->npx = XOR(nullptr, head);

    // If the list was not empty, we must update the old
head's npx
    if (head != nullptr)
    {
        // Old head's npx was: NULL ^ Next
        // We need it to be: NewNode ^ Next

        // 1. Decode the 'next' node of the current head
```

```
        Node *next = XOR(nullptr, head->npx);

        // 2. Update head's npx
        head->npx = XOR(newNode, next);
    }

    head = newNode;
    cout << data << " inserted at the beginning." << endl;
}
…..
        case 1:
            cout << "Enter data to insert: ";
            cin >> data;
            insertAtBeginning(data);
            break;
```

Program Analysis:

```
    Node *newNode = (Node *)malloc(sizeof(Node));
```

→ ***The size of newNode will be equal to the size of a Node structure and newNode is declared as a pointer to Node structure.***

→ ***The size of a pointer (newNode) is typically 8 bytes on most modern 64 − bit systems, regardless of the size of the structure it points to.***

→ ***The size of the Node structure itself is determined by the sum of the sizes of its members***:

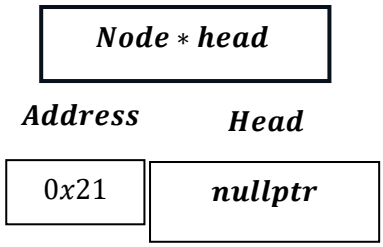
- → ***data(an integer) is typically 4 bytes.***
- → ***npx(a pointer to another Node) is typically 8 bytes.***

→ ***Therefore , the size of the Node structure is typically 12 bytes(4 bytes for data + 8 bytes for npx)*** .

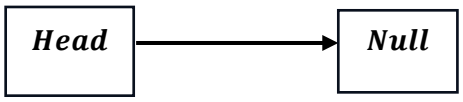
And we already know:

```
#include <cstdint> // Required for uintptr_t (essential for
pointer XOR)
#include <cstdlib> // Required for malloc and free
#include <iostream>

typedef struct XNode
{
    int data;
    struct XNode *npx; // XOR of next and prev pointer
} Node;

Node *head;

Node *XOR(Node *a, Node *b)
{
    return (Node *)((uintptr_t)(a) ^ (uintptr_t)(b));
}

void createEmptyList()
{
    head = nullptr;
}

int main()
{
    createEmptyList();
    return 0;
}
```

Head will point to nullptr , at creation of linked list.



i.e.



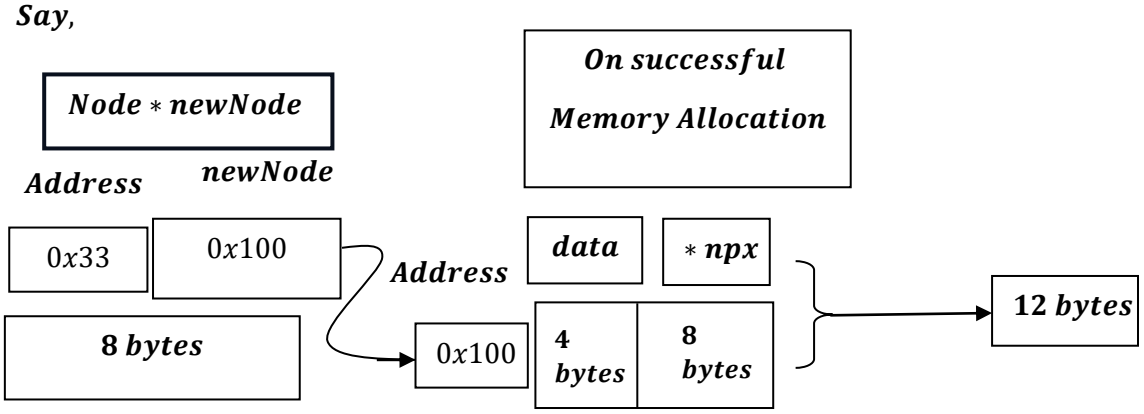
Now coming to insert at beginning :

```
Node *newNode = (Node *)malloc(sizeof(Node));
  if (newNode == nullptr)
  {
      cout << "Error: Memory allocation failed." << endl;
      return;
  }
```

What does it mean?

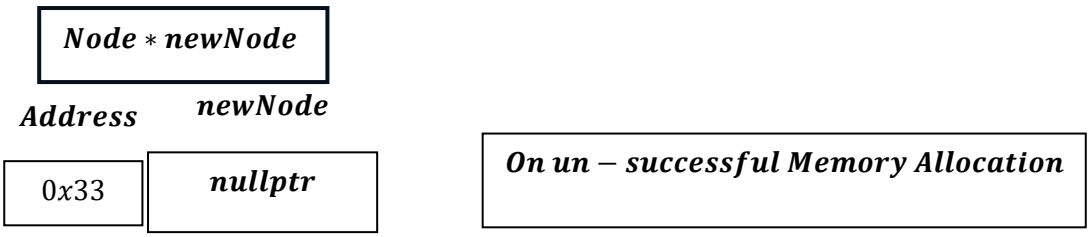
→ ***newNode will try to allocate memory for data i.e. 4 bytes , npx pointer variable i.e. typically 8 bytes, i.e. total 12 bytes, as discussed earlier , the size of a pointer (newNode) is typically 8 bytes on most modern 64 − bit systems, regardless of the size of the structure it points to.***

Say,



But on Un − Successful memory allocation:

→ if the allocation fails due to insufficient memory or other reasons, malloc will return a null pointer.



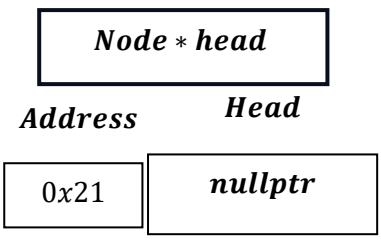
Now, if the value of the pointer variable newNode is nullptr, then:

One get output : `Error: ***Memory Allocation Failed as a message***.`

Now,

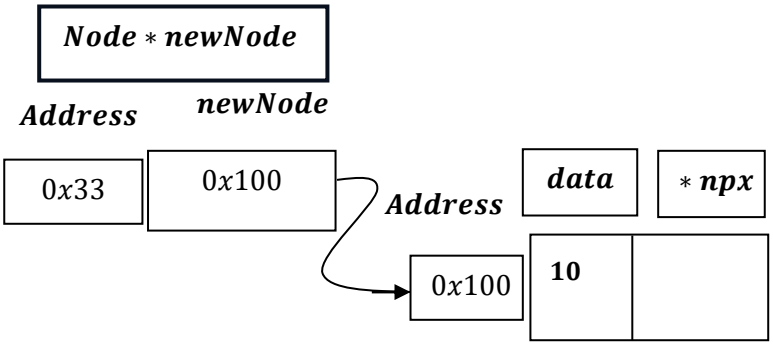
```
case 1:
        cout << "Enter data to insert: ";
        cin >> data;
        insertAtBeginning(data);
        break;
```

Say , data = 10 and currently head is pointed at nullptr.



```
    newNode->data = data;
```

Say,

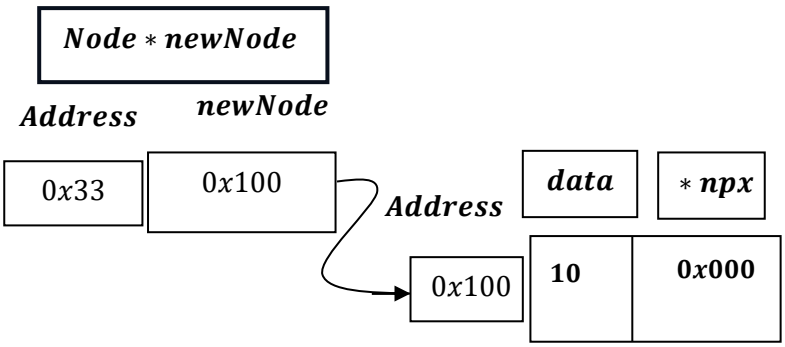



```
Node *XOR(Node *a, Node *b)
{
    return (Node *)((uintptr_t)(a) ^ (uintptr_t)(b));
}
    newNode->npx = XOR(nullptr, head);
```

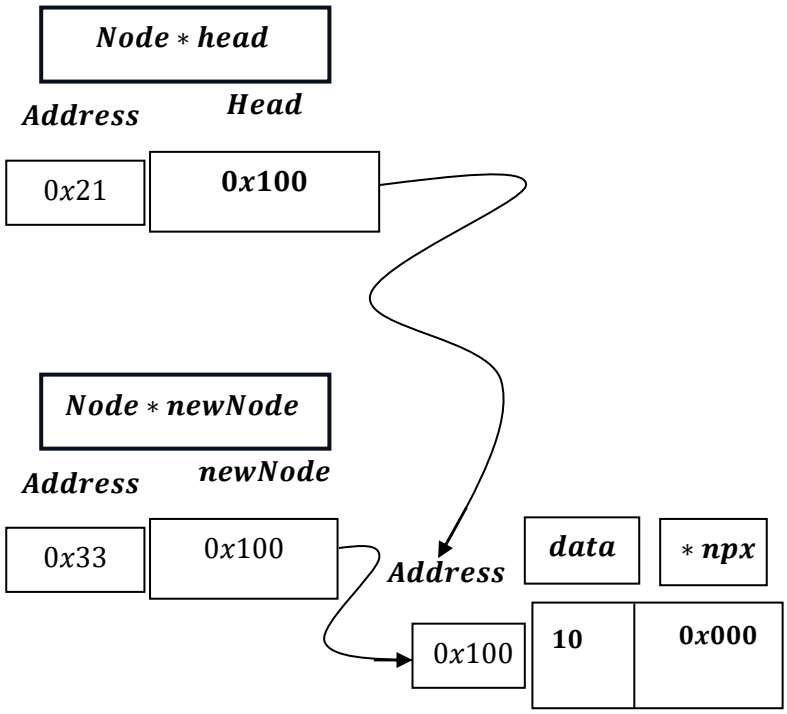
Represent : $NULL \rightarrow 0x000$

Current Head: $NULL \rightarrow 0x0000$

Hence, $newNode \rightarrow npx = XOR(NULL: 0x000 \oplus HEAD: 0x000) = 0x000$



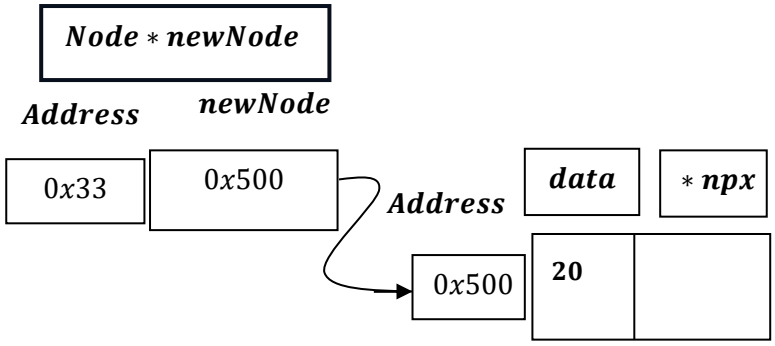
```
  head = newNode;
```

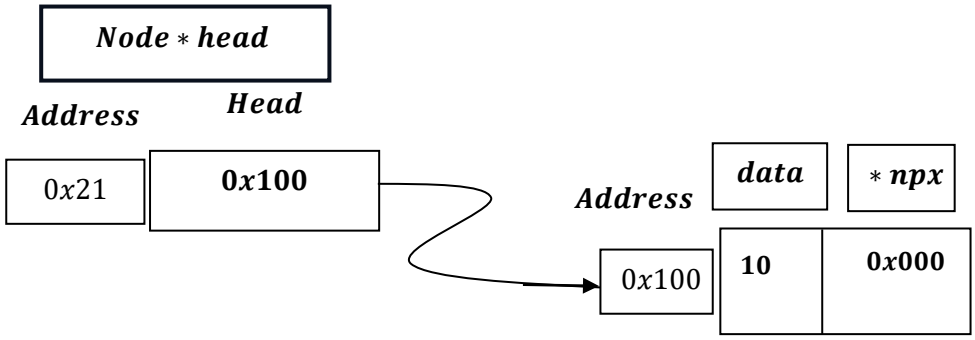


When Head is not NULL!

```
newNode->data = data;
 newNode->npx = XOR(nullptr, head);
if (head != nullptr)
    {
        Node *next = XOR(nullptr, head->npx);

        // 2. Update head's npx
        head->npx = XOR(newNode, next);
    }
  head = newNode;
    cout << data << " inserted at the beginning." << endl;
}
```

Say, next data we want to : **20**.



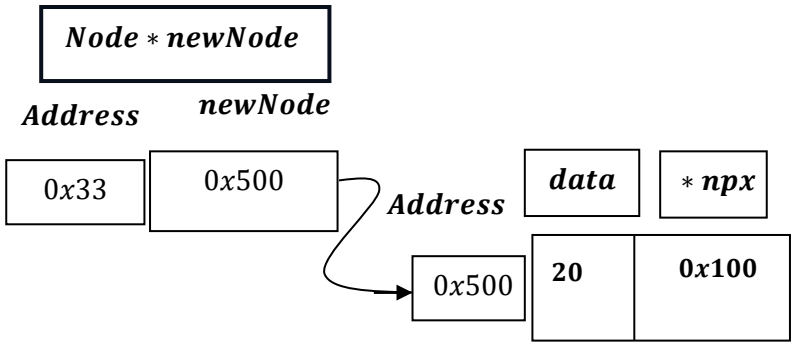


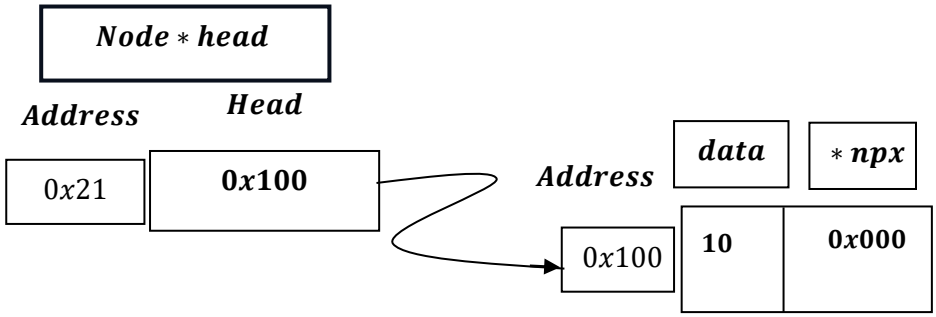

```
Node *XOR(Node *a, Node *b)
{
    return (Node *)((uintptr_t)(a) ^ (uintptr_t)(b));
}
    newNode->npx = XOR(nullptr, head);
```

Represent : $NULL \rightarrow 0x000$

Current Head: $0x100$

Hence, $newNode \rightarrow npx = XOR(NULL: 0x000 \oplus HEAD: 0x100) = 0x100$



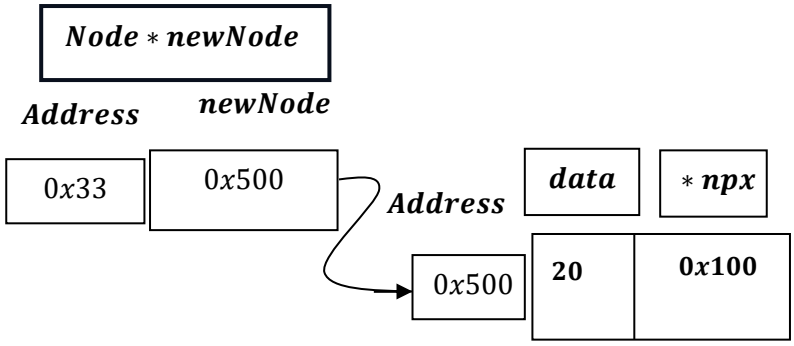


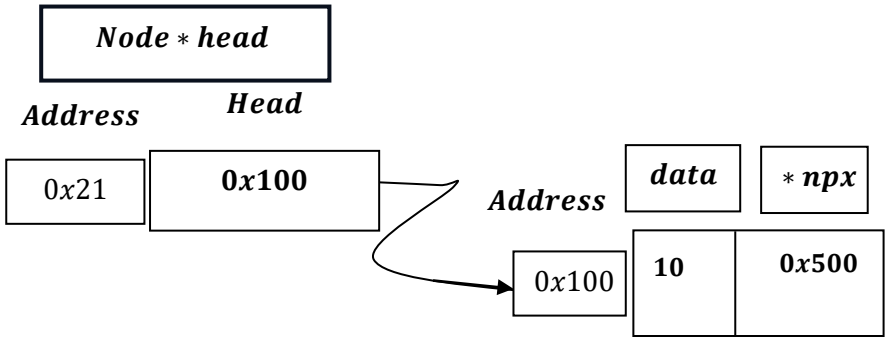

```
        Node *next = XOR(nullptr, head->npx);
```

$Node * next = XOR(NULL: 0x000, Head \rightarrow NPX: 0x000); \Longrightarrow 0x000 \oplus 0x000 = 0x000.$

```
        head->npx = XOR(newNode, next);
```

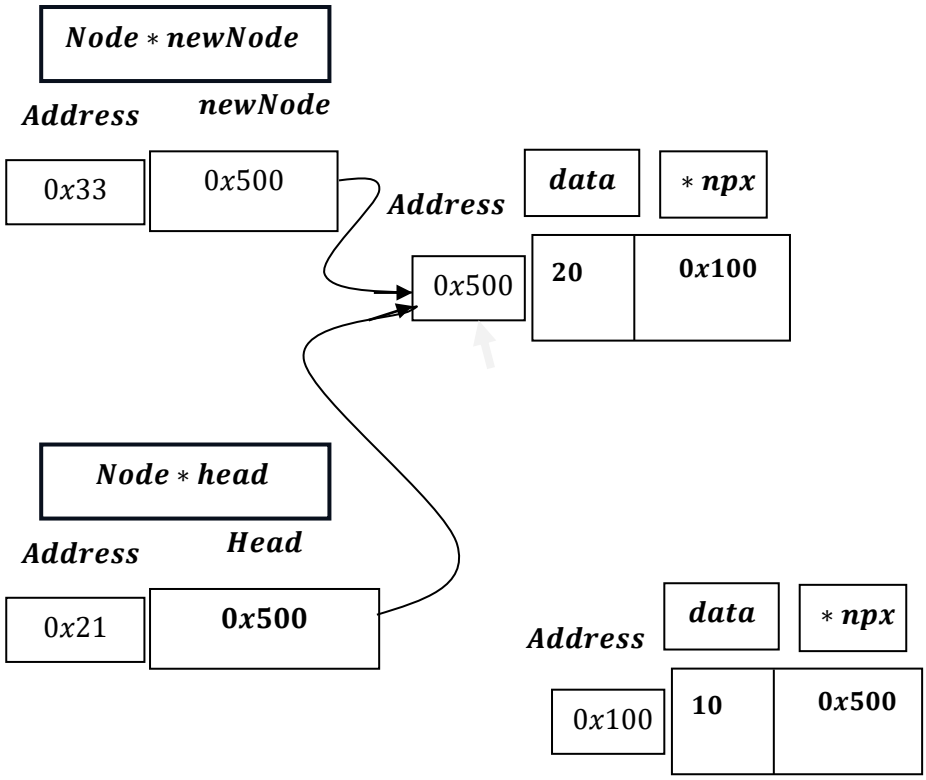
$Head \rightarrow NPX = XOR(newNode: 0x500, next: 0x000) = 0x500 \oplus 0x000 = 0x500.$





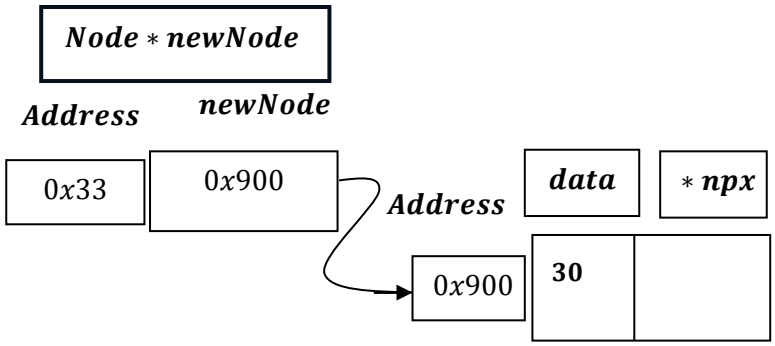

```
head = newNode;
```

$\boldsymbol{head = newNode = 0x500}$;



***Say, we want to enter another data*: 30.**

```
newNode->data = data;
 newNode->npx = XOR(nullptr, head);
if (head != nullptr)
    {
        Node *next = XOR(nullptr, head->npx);

        // 2. Update head's npx
        head->npx = XOR(newNode, next);
    }
  head = newNode;
    cout << data << " inserted at the beginning." << endl;
}
```

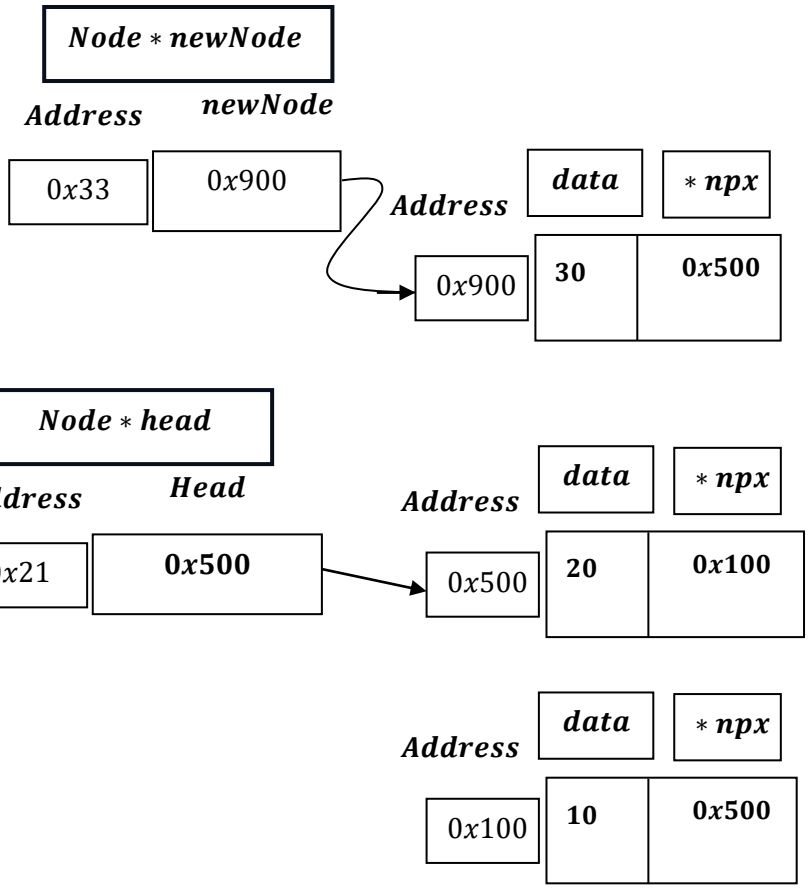



```
Node *XOR(Node *a, Node *b)
{
    return (Node *)((uintptr_t)(a) ^ (uintptr_t)(b));
}
    newNode->npx = XOR(nullptr, head);
```

Represent : $NULL \rightarrow 0x000$

Current Head: $0x500$

Hence, newNode $\rightarrow npx = XOR(NULL: 0x000 \oplus HEAD: 0x500) = 0x500$



```
Node *next = XOR(nullptr, head->npx);
```

$Node * next = XOR(NULL: 0x000\ , Head \rightarrow NPX: 0x100); \Longrightarrow 0x000 \oplus 0x100 = 0x100.$

```
head->npx = XOR(newNode, next);
```

$Head \rightarrow NPX = XOR(newNode: 0x900\ , next: 0x100) = 0x900 \oplus 0x100 = 0x800.$

$0x900 = \ 1001\ 0000\ 0000_2$

$0x100 = \ 0001\ 0000\ 0000_2$

$XORing\ we\ get:$

$1001\ 0000\ 0000_2$

$\oplus$

$0001\ 0000\ 0000_2$

$- - - - - - - - - - - - - -$

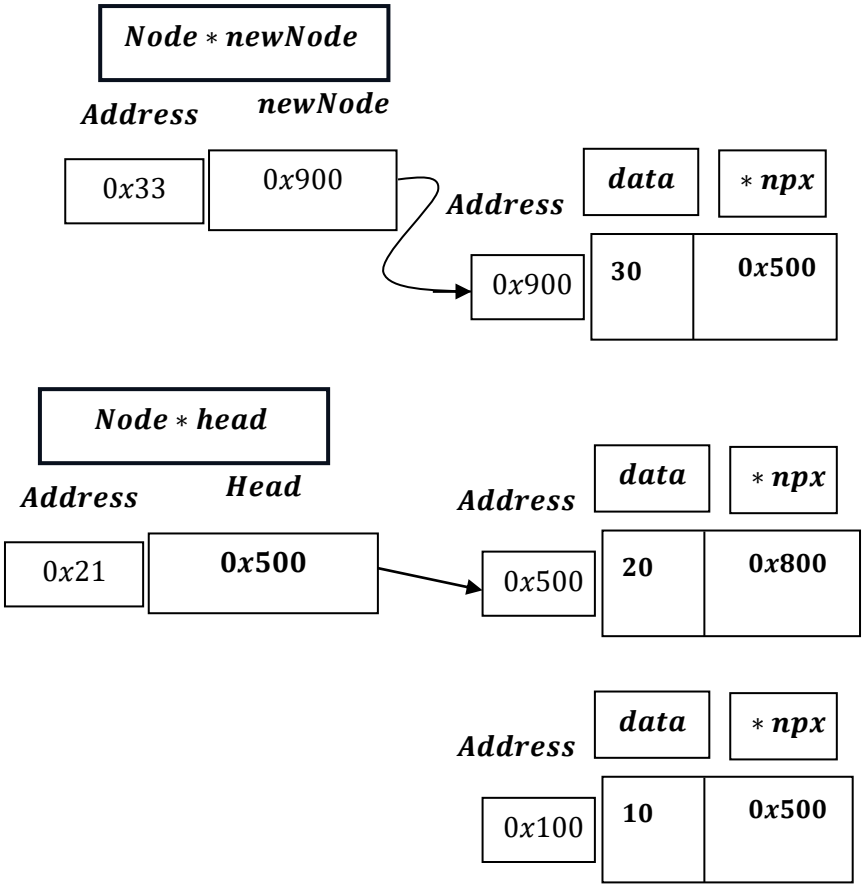
$1000\ \ 0000\ 0000_2$

$- - - - - - - - - - - - - -$

$A[INPUT]$	$B[INPUT]$	$XOR(\oplus)$
0	0	0
0	1	1
1	0	1
1	1	0

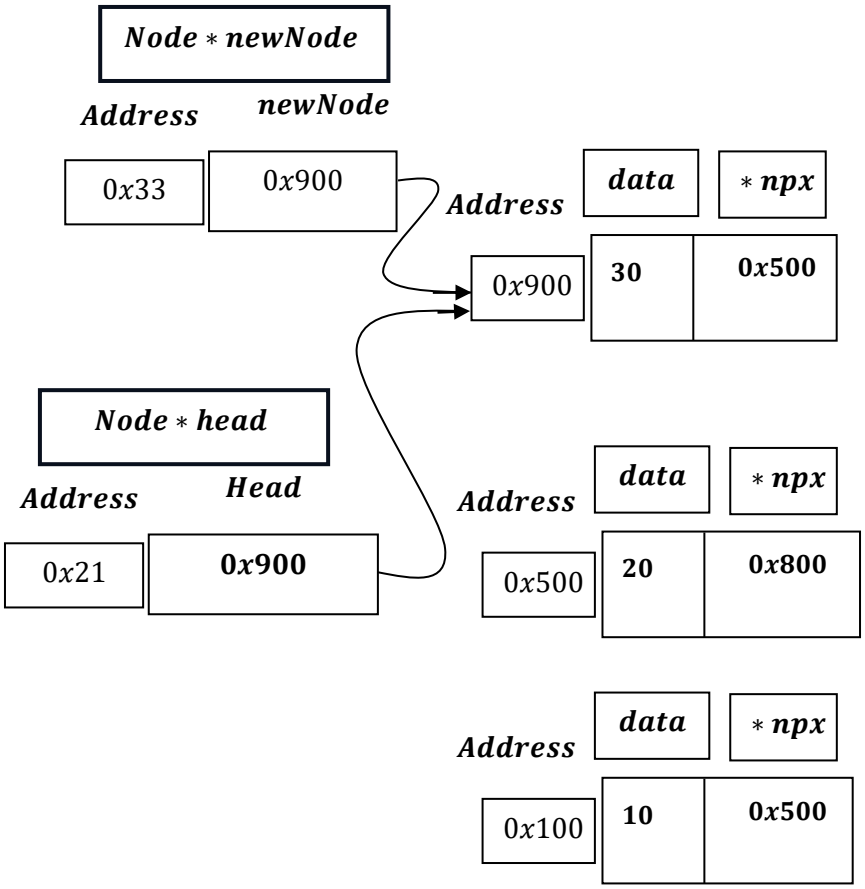
$i.e.\ in\ 4\ bits: 1001_2 \ \oplus 0001_2 = 1000_2 = 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 0 \times 2^0 = 8_{10}.$

$\Longrightarrow 0x900 \oplus 0x100 = 0x800.$

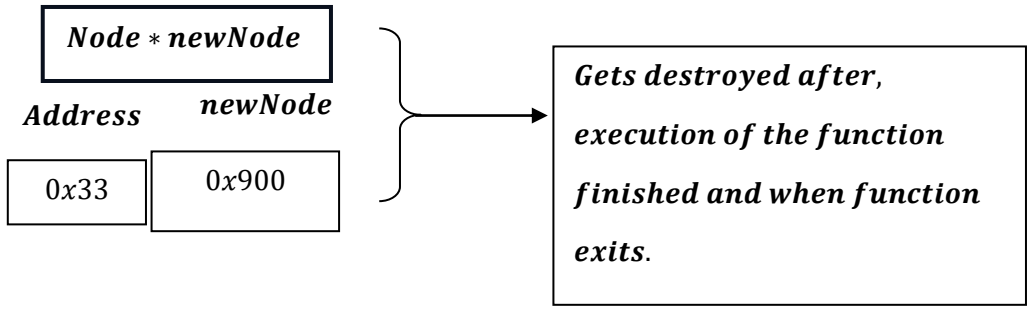



```
head = newNode;
```

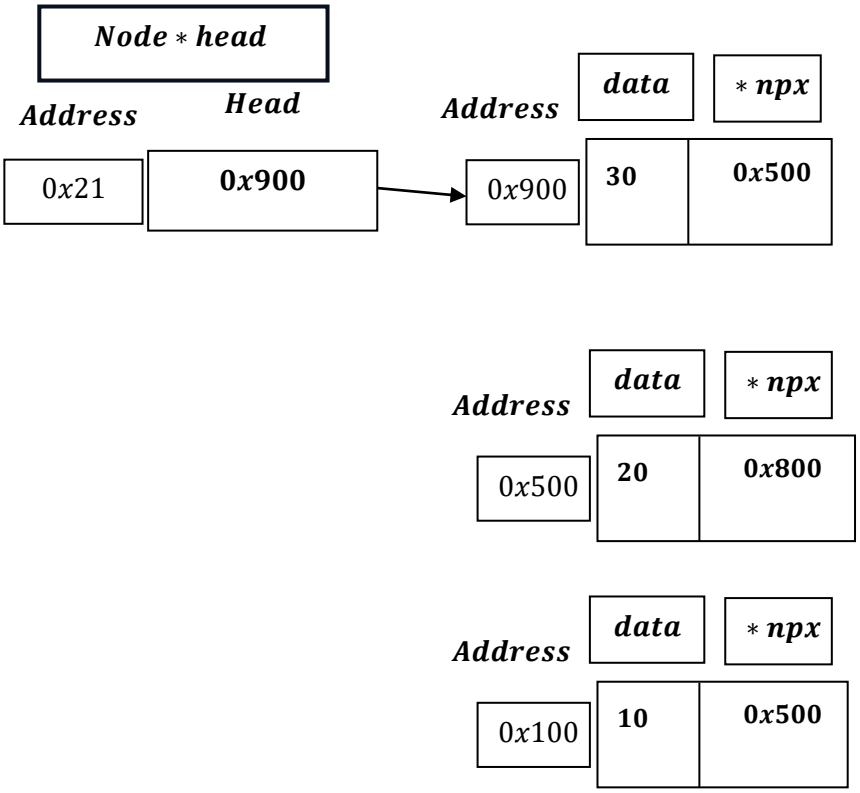
$\boldsymbol{head = newNode = 0x900}$.



After Function ends , temporary pointer Node * newNode gets destroyed:



And , we are left with: –



Therefore, what we have observed, here:

***Node*: *A*(0*x*100)**

***data*: 10**

npx*: *NULL* ⊕ *HEAD

= 0*x*000 ⊕ 0*x*000

= 0*x*000

$Node: A(0x500)$

$data: 20$

$npx =$

$NULL \oplus B$

$= 0x000 \oplus 0x100$

$= 0x100$

$Node: B(0x100)$

$data: 10$

$npx =$

$(NULL \oplus B) \oplus A$

$(0x000 \oplus 0x000) \oplus 0x500$

$= 0x000 \oplus 0x500$

$= 0x500$

$Node: A(0x900)$

$data: 30$

$npx =$

$NULL \oplus B$

$= 0x000 \oplus 0x500$

$= 0x500$

$Node: B(0x500)$

$data: 20$

$npx =$

$(NULL \oplus A) \oplus C$

$= (0x000 \oplus 0x900) \oplus 0x100$

$= 0x900 \oplus 0x100$

$= 0x800$

$Node: C(0x100)$

$data: 10$

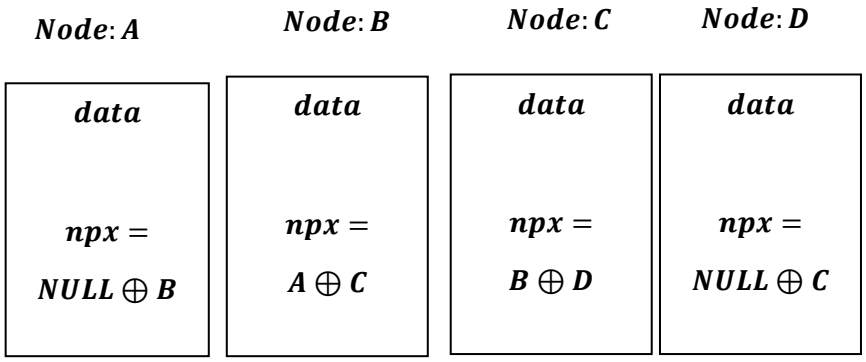
$npx =$

$NULL \oplus B$

$0x000 \oplus 0x500$

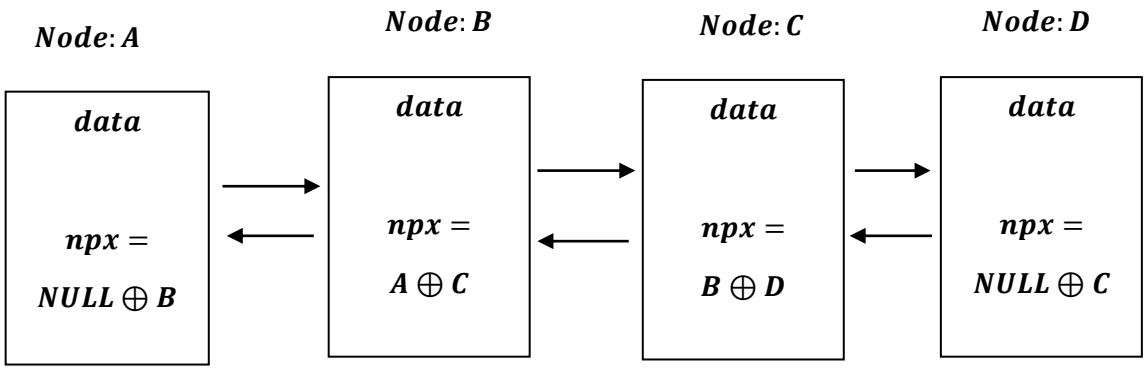
$= 0x500$

Similarly,



- ***There is no separate prev pointer and no separate next pointer stored.***
- ***The npx pointer by itself is not ``a pointer to the previous node`` and not ``a pointer to the next node``.***
- ***It's just a mixed value: the bitwise XOR of two addresses.***

Hence , by XORing only we can achieve:



- ***In C++, ``npx``, is not functioning as a pointer. It is functioning as a Storage Container.***
- ***We declare it as Node * npx only to tell the compiler: "Reserve 8 bytes of space here, because I plan to turn this numbers back into a pointer eventually."***
- ***It is effectively an Integer masquerading as a Pointer.***

The "Envelope" Analogy

Imagine a normal Linked List is a treasure hunt.

- ***Normal Pointer*: *You open an envelope, and inside is a slip of paper that says*: "*Go to House* #100." *You go there, and you find the house.***

- ***XOR Pointer*: *You open the envelope, and inside is a slip of paper that says*: "800".**

    - ***If you try to go to*** House #800, ***you will find an empty field or a cliff edge (Crash).***

    - ***Instead, you are supposed to take your current house number (100) and do math*: 800 *XOR* 100 = 900.**

"***Ah*! *The real destination is House* #900."

In a standard sense, the npx pointer violates the logical definition of a pointer.

- ***Standard Definition*: *A pointer holds the memory address of a specific, valid object.***

- ***XOR Definition*: *The npx variable holds a math result (a number) that does not point to any valid memory location.***

Why does the compiler allow this violation?

C++*: *It assumes you know what you are doing. It allows you to store any number in a pointer variable, as long as you cast it correctly. It only checks validity when you try to dereference (−> or ∗) it.

Summary:

- ***Does npx point to a memory address? No. It points to "garbage"*** (***mathematically calculated noise***).

- ***Does it violate pointer properties? Yes, it violates the property of "Dereferencability." You cannot look inside npx directly.***

- ***Why do we do it? To save space. We sacrifice the "safety" and "direct access" of a pointer to save 50% of our memory usage.***

Memory Efficient Doubly Linked List - Insert At Beginning [Working Summary]:

1. Allocation & Setup

- **Action:** The function allocates memory for a newNode using malloc.
- **Safety:** It checks if the allocation failed (nullptr).
- **Data:** It assigns the input data to the new node.

2. Establishing the New Head's Links

- **Goal:** Determine the npx value for the newNode.
- **Logic:** Since this node is being inserted at the **very beginning**:
  - Its **Previous** node is nullptr.
  - Its **Next** node is the current head.
- **Code:** newNode->npx = XOR(nullptr, head);

3. Updating the Old Head (The "Stitching")

If the list was not empty (head != nullptr), the old head needs to know that it is no longer the first node. Its "Previous" link must change from nullptr to newNode.

- **Step A: Decode the Neighbor**
  - We cannot just write to head->npx directly because it contains a compressed value (Prev ⊕ Next). We need to isolate the Next part first.
  - **Math:** next = nullptr ⊕ head->npx (This recovers the address of the 2nd node).
- **Step B: Re-encode with New Previous**
  - Now we calculate the new XOR checksum for the old head.
  - **New Previous:** newNode
  - **Next:** next (unchanged)
  - **Code:** head->npx = XOR(newNode, next);

4. Updating the Global Pointer

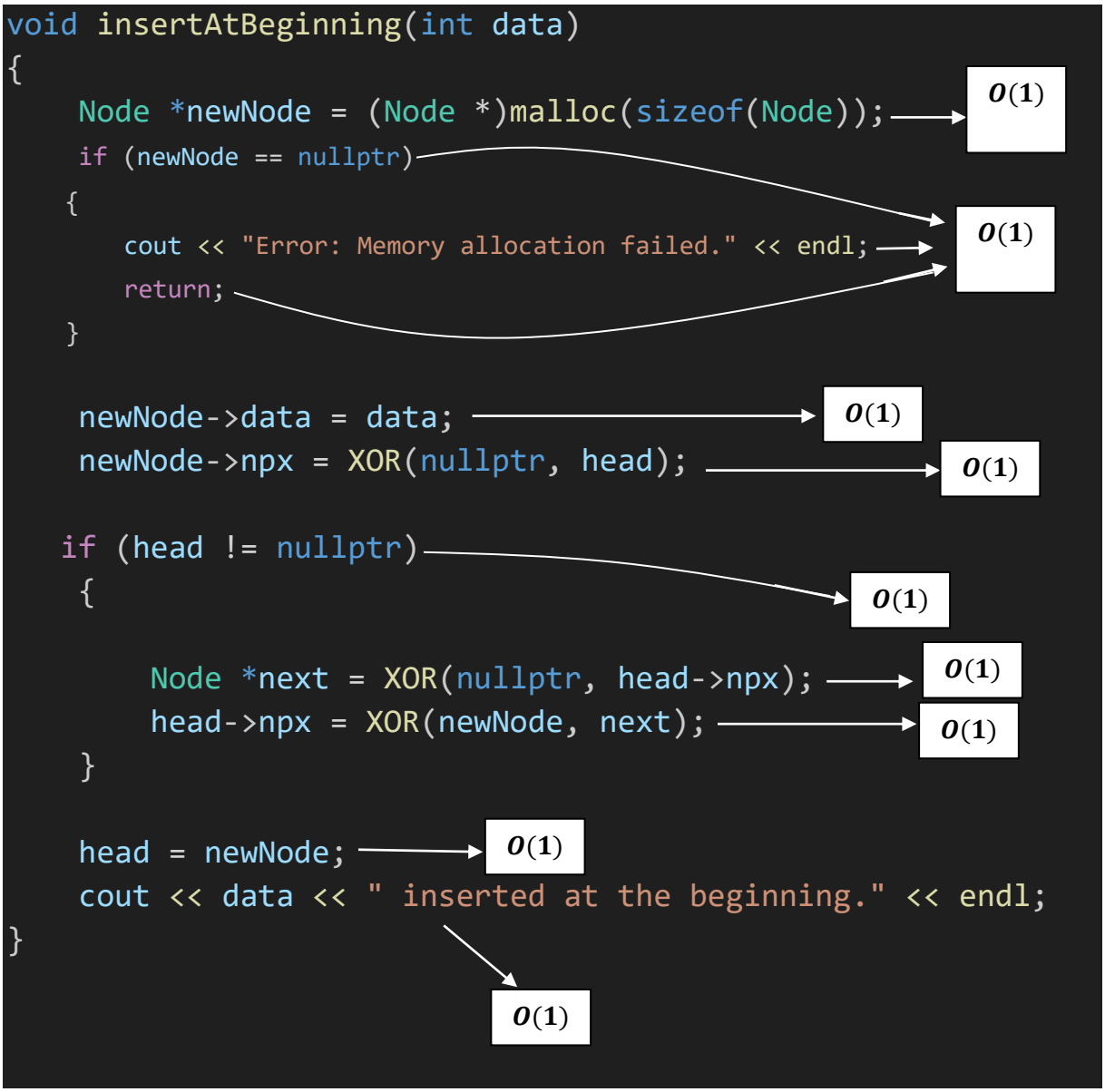
- **Action:** Finally, the global head pointer is updated to point to the newNode, officially making it the start of the list.

Visual Logic Summary

Scenario: Inserting **A** before **B**.

1. **Create A.**
2. **Calculate A->npx:** $NULL \oplus Address(B)$
3. **Update B (Old Head):**
   - *Old B->npx:* $NULL \oplus Address(C)$
   - *New B->npx:* $Address(A) \oplus Address(C)$
4. **Set Head = A.**

Time Complexity of the program



$$\boldsymbol{Best\ Case = Worst\ Case = Average\ Case = O(1)}$$

Memory Efficient Doubly Linked List	*Cases*	*Time Complexity*
InsertAt Beginning	1. *Best Case*	$\Omega(1)$
	2. *Worst Case*	$O(1)$
	3. *Average Case*	$\Theta(1)$

****